Where Should a Robot Swarm See Its Neighbors? Communication-Range Transfer in a GNN Coverage Policy

Loïc Poisson **Harold Gardon** **Claire Fitussi**
Stanford University
{lpoisson, gardon21, cfitussi}@stanford.edu

1 Application domain

Problem. In *coverage control*, N robots must spread over an area so that their sensors watch the regions that matter, described by an *importance density field* (IDF) ϕ that the robots do not know in advance [Cortés et al., 2004]. Each robot only sees a 64×64 m sensor footprint and can only exchange messages with robots within its *communication range* R; there is no central computer. The communication network is a graph $G_R = (V, E_R)$ with one node per robot and an edge (i, j) whenever $\|p_i - p_j\| \leq R$. This makes decentralized control a natural graph learning problem: every robot must decide what to send to its neighbors and how to combine what it receives [Tolstaya et al., 2020].

Dataset. We use the **CoverageControl** library [KumarRobotics, 2024] (GPL-3.0, `pip install coverage_control`), released with the LPAC paper [Agarwal et al., 2025]. It provides (i) a simulator of 32 robots in a 1024×1024 m world, with IDFs drawn as mixtures of 32 random Gaussians; (ii) the published LPAC checkpoint and the 11 evaluation environments it ships with; and (iii) scripts that generate imitation data by running a *clairvoyant* centroidal-Voronoi (CVT) controller that knows ϕ and all positions. One training sample is the state of all 32 robots at one time step: each robot's four 32×32 input maps, its position, the communication graph, and the clairvoyant's velocity as target. As in the paper, a training set has 100,000 samples; we also saved 40 new evaluation environments. Everything we use is public; our generated environments, data-generation configurations and code will be released in our repository[1] before the final submission.

Task and metric. The learned policy maps each robot's local observations and messages to a velocity, i.e. node-level regression on G_R, trained by imitation (mean squared error to the clairvoyant). We evaluate it in closed loop for 600 steps and report the paper's metric: the coverage cost $J = \int \phi(q) \min_i \|q - p_i\|^2 \, dq$ averaged over the episode, divided by the clairvoyant's cost on the same environment (lower is better; centralized CVT scores about 2.5). Our protocol varies the *test-time* range $R \in \{128, 256, 512, 768, 1024\}$ m, reports means over environments with paired differences, and uses three training seeds to set noise thresholds.

Why this dataset. LPAC is a recent published GNN policy whose authors report an unexplained anomaly: performance degrades when R grows (their Sec. VI-H). The simulator controls graph density exactly, the model is written in PyTorch Geometric [Fey and Lenssen, 2019], and a full training run takes 2–4 h on one free Stanford FarmShare GPU.

CS224W: Machine Learning with Graphs, Stanford University, Autumn 2026. Project proposal (type 1: real-world application).

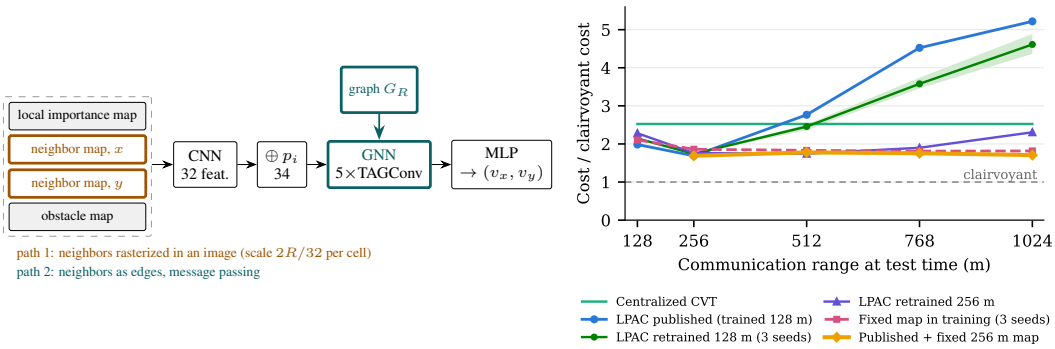


Figure 1: **Left:** LPAC on one robot. Neighbors enter twice: drawn in a 32×32 neighbor map read by the CNN (path 1) and as edges of G_R read by the GNN (path 2). **Right:** cost (lower is better), mean over the 11 released environments. Models trained at 128 m collapse at larger test ranges; drawing the neighbor map at a fixed 256 m scale removes the collapse, at test time or during training on mixed ranges.

2 Graph ML techniques

Model: LPAC [Agarwal et al., 2025]. Each robot runs the same network (Fig. 1, left). A CNN encodes four 32×32 maps into 32 features, the robot position is appended, five TAGConv layers [Du et al., 2017] with $K = 3$ exchange information over G_R, and an MLP outputs the velocity. With node features X, adjacency A and degree matrix D,

$$X^{(\ell+1)} = \mathrm{ReLU}\Big(\textstyle\sum_{k=0}^{K} \big(D^{-1/2} A D^{-1/2}\big)^k X^{(\ell)} W_k^{(\ell)} \Big), \tag{1}$$

the GCN propagation [Kipf and Welling, 2017] applied up to 15 times in total. Two of the CNN's input channels form a *neighbor map*: robot j within range is written into cell $c_{ij} = \mathrm{round}\big((p_j - p_i)\frac{32}{2R} + 15.5\big)$ with value $(p_j - p_i)/R$, values summed per cell. GNNs suit this problem because message passing is exactly local communication: the policy is decentralized, permutation-equivariant, and independent of team size [Tolstaya et al., 2020].

Preliminary findings. (1) *Reproduction.* Retrained, LPAC matches the published model (1.74 vs 1.69 at 256 m). Trained separately at 128, 256, 512 and 1024 m and tested at its own range, it beats both CVT baselines at every range, as in the paper, but the drop beyond 512 m reported there does not appear in our runs (2.15 / 1.76 / 1.79 / 1.80, one seed). (2) *Transfer collapse.* The model trained at 128 m reaches 5.22 at 1024 m, worse than both baselines in all 11 environments; three retrained seeds reproduce this (Fig. 1). (3) *Cause.* At 1024 m, capping G_R to 4 neighbors leaves the cost at 5.11, while drawing only the neighbor map at a fixed 256 m scale brings it to 1.71, the 256 m performance (paired $+0.02 \pm 0.16$), on every model and 40 unseen environments; TAGConv outputs keep a high effective rank [Roy and Vetterli, 2007], ruling out over-smoothing. (4) *Heterogeneous teams.* When a fraction of robots hears at 1024 m and the rest at 128 m, the cost grows convexly with that fraction, while the fixed-scale map keeps it flat (1.7–2.1).

Planned techniques. Our question is *where neighbor geometry should live, and at which scale*: in a rasterized image read by a CNN, or on the edges of the graph. Under identical data and budgets we compare: (a) inference-time interventions on each input path; (b) training-time remedies: mixed communication ranges, a domain-randomization baseline [Tobin et al., 2017] (1.9 at 1024 m but 0.3 worse at 256 m), the robot's own range as an input (no gain over mixed ranges), and a fixed-scale map used during training, our best recipe so far (three seeds: 1.86 at 256 m, 1.81–1.83 up to 1024 m, $+0.12 \pm 0.05$ behind LPAC at 256 m); (c) graph-native encoders that drop the map and put the relative position $p_j - p_i$, scaled by a fixed length, on the edges, as in related swarm policies [Muthusamy et al., 2024, Owerko et al., 2025]: GATv2 [Brody et al., 2022] underfits, and a PointNet edge layer [Qi et al., 2017] before the TAGConv stack degrades little with range (2.01 to 2.58, three seeds) but fails when trained on mixed ranges, since its inputs grow with R; we will test bounded edge features (unit direction, saturating distance); (d) heterogeneous teams, with one specialist model per range. Success: within 0.05 of LPAC at 256 m, flat up to 1024 m, with no test-time fix (three seeds).

[1] `https://github.com/loicpoisson/lpac-communication-range` (made public before the final submission).

References

Saurav Agarwal, Ramya Muthukrishnan, Walker Gosrich, Vijay Kumar, and Alejandro Ribeiro. LPAC: Learnable perception-action-communication loops with applications to coverage control. *IEEE Transactions on Robotics*, 41, 2025. arXiv:2401.04855.

Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations*, 2022.

Jorge Cortés, Sonia Martínez, Timur Karataş, and Francesco Bullo. Coverage control for mobile sensing networks. *IEEE Transactions on Robotics and Automation*, 20(2):243–255, 2004.

Jian Du, Shanghang Zhang, Guanhang Wu, José M. F. Moura, and Soummya Kar. Topology adaptive graph convolutional networks. *arXiv preprint arXiv:1710.10370*, 2017.

Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

KumarRobotics. CoverageControl: A library for coverage control with robot swarms, release v1.3.1. https://github.com/KumarRobotics/CoverageControl, 2024.

Shreyas Muthusamy, Damian Owerko, Charilaos I. Kanatsoulis, Saurav Agarwal, and Alejandro Ribeiro. Generalizability of graph neural networks for decentralized unlabeled motion planning. *arXiv preprint arXiv:2409.19829*, 2024.

Damian Owerko, Frederic Vatnsdal, Saurav Agarwal, Vijay Kumar, and Alejandro Ribeiro. MAST: Multi-agent spatial transformer for learning to collaborate. *arXiv preprint arXiv:2509.17195*, 2025.

Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems*, 2017.

Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *European Signal Processing Conference (EUSIPCO)*, 2007.

Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

Ekaterina Tolstaya, Fernando Gama, James Paulos, George Pappas, Vijay Kumar, and Alejandro Ribeiro. Learning decentralized controllers for robot swarms with graph neural networks. In *Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 671–682, 2020.